



Mineração de Dados

Módulo 2: Pré-processamento de Dados

25 de junho de 2026

Sumário

1	Objetivos de Aprendizagem	4
2	Qualidade de Dados	4
3	Tipos de Dados	5
4	Dados Ausentes (Missing Values)	6
4.1	Mecanismos de Ausência	6
4.2	Técnicas de Imputação	7
4.2.1	Imputação pela Média, Mediana ou Moda	7
4.2.2	Hot-Deck Imputation	7
4.2.3	Imputação por KNN	7
4.2.4	Imputação Iterativa – MICE	8
4.2.5	Imputação por Modelos Baseados em ML	8
5	Detecção e Tratamento de Outliers	9
5.1	Métodos Univariados	9
5.1.1	Z-Score	9
5.1.2	IQR (Interquartile Range)	9
5.1.3	Z-Score Modificado (MAD)	9
5.2	Métodos Multivariados e Baseados em ML	10
5.3	Como Tratar Outliers	10
6	Normalização e Escalonamento	10
6.0.1	Min-Max Normalization	11
6.0.2	Z-Score Standardization	11
6.0.3	Robust Scaler	11
6.0.4	MaxAbs Scaler	11
6.0.5	Transformação Logarítmica	11
7	Seleção de Atributos (Feature Selection)	11
7.1	Métodos de Filtro	12
7.2	Métodos Wrapper	12
7.3	Métodos Embutidos (Embedded)	13
8	Redução de Dimensionalidade	13
8.1	PCA – Análise de Componentes Principais	13
8.2	t-SNE e UMAP	14
8.3	Autoencoders para Redução	14
9	Balanceamento de Classes	14
9.0.1	Oversampling	14
9.0.2	Undersampling	15
10	Estado da Arte	15

11 Exercícios	16
12 Referências Bibliográficas	16

1 Objetivos de Aprendizagem

Ao final deste módulo, o estudante será capaz de:

1. **✓ Compreender** os conceitos de qualidade de dados e suas seis dimensões principais (acurácia, completude, consistência, temporalidade, validade e unicidade), reconhecendo sua importância no processo KDD.
2. **✓ Distinguir** os tipos de dados (escalas nominal, ordinal, intervalar e razão; dados estruturados e não estruturados; séries temporais e dados espaciais) e selecionar as técnicas de pré-processamento adequadas para cada tipo.
3. **✓ Aplicar** técnicas de tratamento de dados ausentes (imputação pela média/mediana, KNN, MICE) identificando o mecanismo de ausência (MCAR, MAR, MNAR) em cada situação.
4. **✓ Identificar** e tratar outliers usando métodos univariados (Z-score, IQR) e multivariados (Isolation Forest, LOF, One-Class SVM), escolhendo a estratégia de tratamento mais adequada.
5. **✓ Aplicar** técnicas de normalização e escalonamento (Min-Max, Z-score, Robust Scaler) justificando a escolha com base nas características dos dados e no algoritmo a ser utilizado.
6. **✓ Construir** pipelines de seleção de atributos usando métodos de filtro (correlação, informação mútua), wrapper (RFE) e embutidos (LASSO, importância em árvores).
7. **✓ Avaliar** estratégias de redução de dimensionalidade (PCA, t-SNE, UMAP) e balanceamento de classes (SMOTE, ADASYN), selecionando a abordagem mais adequada ao problema.
8. **✓ Construir** pipelines completos de pré-processamento com sklearn Pipeline e ColumnTransformer, garantindo que transformações sejam aplicadas corretamente em dados de treino e teste.

2 Qualidade de Dados

O pré-processamento de dados é frequentemente citado como a etapa mais crítica e mais demorada de um projeto de mineração de dados. Pesquisas indicam que 60 a 80% do esforço total de um projeto de ciência de dados é dedicado à limpeza e preparação dos dados. Dados de baixa qualidade não apenas comprometem o desempenho dos modelos, mas podem levar a conclusões completamente equivocadas e a decisões de negócio prejudiciais.

Qualidade de Dados

Qualidade de dados é a medida em que os dados satisfazem os requisitos de uso para os quais foram coletados. Dados de alta qualidade são aqueles que representam fielmente a realidade que descrevem e que podem ser usados de forma confiável para

as análises pretendidas.

i Regra 80-20 do Pré-processamento

Na prática de projetos de ciência de dados, a **regra 80-20** é um consenso informal: 80% do tempo é gasto em pré-processamento e apenas 20% em modelagem propriamente dita. Isso reflete a realidade de dados do mundo real, que quase nunca chegam limpos e estruturados. Investir bem nesta etapa é fundamental para o sucesso do projeto.

As seis dimensões principais de qualidade de dados são:

1. **Acurácia:** Os dados refletem corretamente a realidade? Exemplo de problema: uma base de clientes com CEP “99999-999” (inválido) ou data de nascimento “01/01/1900” (implausível).
2. **Completude:** Todos os dados necessários estão presentes? Exemplo: formulários de cadastro com campo “renda mensal” deixado em branco por 40% dos respondentes.
3. **Consistência:** Os dados são coerentes entre si e com outras fontes? Exemplo: um registro indica que o cliente é do sexo feminino mas o pronome usado em outros campos é masculino; ou a data de entrega é anterior à data de pedido.
4. **Temporalidade:** Os dados estão atualizados? Exemplo: endereços de clientes coletados há 5 anos podem estar desatualizados; preços de produtos sem data de vigência.
5. **Validade:** Os dados estão dentro dos domínios e formatos esperados? Exemplo: campo “idade” com valor negativo ou maior que 150; CPF com 11 dígitos mas matematicamente inválido.
6. **Unicidade:** Não há duplicatas indevidas? Exemplo: o mesmo cliente cadastrado duas vezes com pequenas variações no nome (“João Silva” e “Joao Silva”); registros idênticos em tabelas de transações.

3 Tipos de Dados

A classificação dos tipos de dados é fundamental para escolher as técnicas de pré-processamento e os algoritmos adequados. A escala de medida proposta por Stevens (1946) ainda é o referencial mais utilizado:

Além das escalas de medida, é importante distinguir:

- **Dados contínuos vs. discretos:** Contínuos assumem qualquer valor em um intervalo (altura, temperatura); discretos assumem valores enumeráveis (número de filhos, contagem de cliques).
- **Dados estruturados vs. não estruturados:** Estruturados seguem um esquema tabular rígido (planilhas, tabelas SQL); não estruturados incluem texto livre, imagens, áudio e vídeo, requerem extração de features antes da mineração tradicional.

Tabela 1: Escalas de medida de Stevens (1946) e suas características

Escala	Propriedades	Operações das	Válidas	Exemplos
Nominal	Apenas categorias, sem ordem	Igualdade, ordem, moda	contagem	Cor dos olhos, Estado civil, Tipo sanguíneo, CEP
Ordinal	Categorias com ordem, mas intervalos desiguais	Comparação de ordem, mediana, percentis		Grau de satisfação (1–5), Nível de escolaridade, Classificação em corrida
Intervalar	Intervalos iguais, mas zero é arbitrário	Adição, subtração, média, desvio padrão		Temperatura em Celsius, Ano calendário, Escores padronizados
Razão	Intervalos iguais e zero absoluto	Todas as operações, incluindo razões e multiplicações		Peso (kg), Altura (cm), Renda (R\$), Tempo (s)

- **Séries temporais:** Sequências ordenadas no tempo. Exigem técnicas especiais (janelas deslizantes, decomposição de tendência e sazonalidade, modelos ARIMA, LSTM).
- **Dados espaciais:** Dados com componente geográfico (coordenadas, polígonos). Requerem métricas de distância geodésica e técnicas de geostatística.
- **Dados de texto:** Requerem pré-processamento especializado (tokenização, stemming, lematização, remoção de stopwords, TF-IDF, embeddings).

4 Dados Ausentes (Missing Values)

Dados ausentes são um problema ubíquo em aplicações reais. A abordagem correta para tratar valores ausentes depende fundamentalmente do *mecanismo de ausência*, que determina se e como a ausência é relacionada aos dados observados e não observados.

4.1 Mecanismos de Ausência

- **MCAR** (*Missing Completely At Random*): A probabilidade de um valor estar ausente não depende de nenhum dado observado ou não observado. Exemplo: um sensor de temperatura falha aleatoriamente a cada 1000 medições, independentemente do valor medido. Neste caso, a remoção de registros com dados ausentes não introduz viés, mas reduz o poder estatístico.
- **MAR** (*Missing At Random*): A probabilidade de ausência depende de outros dados observados, mas não do valor ausente em si. Exemplo: respondentes com renda mais alta tendem a não informar a renda (mas a ausência depende de outras características observadas, como nível de escolaridade). Técnicas de imputação múltipla (como MICE) são adequadas neste caso.

- **MNAR** (*Missing Not At Random*): A probabilidade de ausência depende do próprio valor ausente. Exemplo: pacientes com valores muito altos de pressão arterial tendem a não comparecer às consultas (e portanto não têm a medição registrada). Este é o cenário mais difícil, qualquer imputação sem modelagem explícita do mecanismo de ausência introduz viés.

⚠ Importância do Diagnóstico do Mecanismo

A escolha da estratégia de tratamento de dados ausentes **deve ser precedida** do diagnóstico do mecanismo de ausência. Remover registros com dados ausentes quando o mecanismo é MAR ou MNAR introduz viés sistemático nos modelos. Testes estatísticos como Little's MCAR test podem auxiliar no diagnóstico.

4.2 Técnicas de Imputação

4.2.1 Imputação pela Média, Mediana ou Moda

A imputação pela estatística central substitui cada valor ausente pela média (para variáveis numéricas simétricas), mediana (para numéricas assimétricas ou com outliers) ou moda (para categóricas).

$$\hat{x}_i = \begin{cases} \bar{x} & (\text{média, se distribuição simétrica}) \\ \text{mediana}(x) & (\text{se distribuição assimétrica}) \\ \text{moda}(x) & (\text{se variável categórica}) \end{cases}$$

Vantagens: Simples, rápido, fácil de implementar.

Desvantagens: Subestima a variância, distorce correlações entre variáveis, trata todos os ausentes de forma idêntica independentemente do contexto.

Quando usar: Poucos valores ausentes ($< 5\%$), mecanismo MCAR, como baseline comparativo.

4.2.2 Hot-Deck Imputation

Cada valor ausente é substituído pelo valor de um doador, uma instância similar selecionada da mesma base de dados. A similaridade pode ser definida por vizinhança geográfica, faixa etária, ou outros critérios de domínio.

Vantagens: Preserva distribuições e relações entre variáveis melhor que a imputação pela média.

Desvantagens: Pode ser computacionalmente custoso; a definição de “similar” é subjetiva.

4.2.3 Imputação por KNN

O KNN Imputer substitui cada valor ausente pela média ponderada dos k vizinhos mais próximos no espaço de features observadas.

Algorithm 1: KNN Imputation

Input: Dataset X com valores ausentes, número de vizinhos k
Output: Dataset X' com valores imputados

```

1 for cada instância  $x_i$  com valores ausentes do
2   Calcular distância  $d(x_i, x_j)$  apenas nas features observadas em ambas as
   instâncias;
3   Selecionar os  $k$  vizinhos mais próximos  $N_k(i) = \{x_{j_1}, \dots, x_{j_k}\}$ ;
4   for cada feature  $f$  ausente em  $x_i$  do
5      $\hat{x}_{i,f} \leftarrow \frac{\sum_{j \in N_k(i)} w_j \cdot x_{j,f}}{\sum_{j \in N_k(i)} w_j}$  onde  $w_j = 1/d(x_i, x_j)$ ;
6   end
7 end
8 return  $X'$ ;

```

Vantagens: Captura relações locais entre instâncias; preserva a estrutura dos dados melhor que imputação global.

Desvantagens: Sensível à escala das features (normalizar antes!); custo $O(n^2)$; não funciona bem com dados muito esparsos.

4.2.4 Imputação Iterativa – MICE

O MICE (*Multiple Imputation by Chained Equations*, Van Buuren, 2018) trata a imputação como um problema de regressão: cada coluna com ausentes é modelada como variável dependente, usando as outras colunas como preditores. O processo é iterado até a convergência.

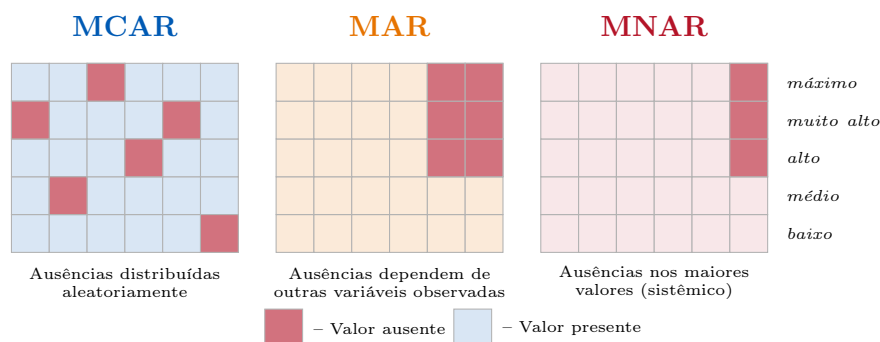
Vantagens: Modela relações multivariadas complexas; produz estimativas de incerteza; robusto para MAR.

Desvantagens: Computacionalmente intensivo; assumem relações paramétricas (versão básica).

4.2.5 Imputação por Modelos Baseados em ML

MissForest (Stekhoven & Buhlmann, 2012) usa Random Forests para imputar: itera entre todas as colunas, treinando uma floresta por coluna e usando as previsões como imputações. GAIN (Yoon et al., 2018) usa GANs para imputação. São as abordagens com melhor desempenho em geral, especialmente com dados complexos.

A figura a seguir ilustra os três padrões de ausência (MCAR, MAR, MNAR) em uma grade de dados:



5 Detecção e Tratamento de Outliers

Outliers são observações que se desviam significativamente do padrão esperado nos dados. Eles podem representar erros de coleta, medição ou inserção de dados, mas também podem ser observações genuínas e informativas (como transações fraudulentas ou eventos raros clinicamente relevantes). A estratégia de tratamento deve ser guiada pela natureza do problema.

5.1 Métodos Univariados

5.1.1 Z-Score

O Z-score mede quantos desvios padrão uma observação está afastada da média:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Considera-se outlier quando $|z_i| > 3$ (equivalente a estar fora do intervalo $\mu \pm 3\sigma$, que cobre 99,7% de uma distribuição normal). Sensível a outliers extremos que distorcem μ e σ .

5.1.2 IQR (Interquartile Range)

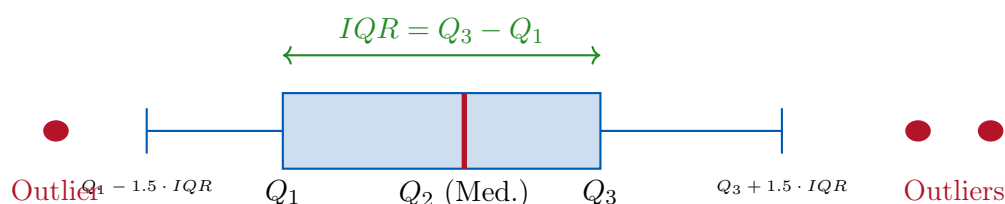
O método IQR é mais robusto, pois usa medidas de posição resistentes:

$$IQR = Q_3 - Q_1$$

$$\text{Outlier se: } x_i < Q_1 - 1.5 \cdot IQR \text{ ou } x_i > Q_3 + 1.5 \cdot IQR$$

O fator 1.5 é o critério de Tukey (1977). Usando 3.0 (critério mais restritivo) identifica apenas outliers extremos.

O boxplot a seguir ilustra os elementos do método IQR:



5.1.3 Z-Score Modificado (MAD)

Usa a Mediana Absolute Deviation (MAD) em vez de σ , sendo extremamente robusto:

$$MAD = \text{mediana}(|x_i - \text{mediana}(x)|)$$

$$M_i = \frac{0.6745 \cdot (x_i - \text{mediana}(x))}{MAD}$$

Outlier se $|M_i| > 3.5$.

5.2 Métodos Multivariados e Baseados em ML

Distância de Mahalanobis: Estende o Z-score para múltiplas dimensões, considerando a correlação entre variáveis: $D_M(x) = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}$. Sensível quando $n \gg p$.

Isolation Forest (Liu et al., 2008): Isola outliers construindo árvores aleatórias. Anomalias são isoladas em menos divisões, resultando em caminhos médios mais curtos. Eficiente e eficaz para dados de alta dimensionalidade.

LOF (*Local Outlier Factor* - Breunig et al., 2000): Mede a densidade local de cada ponto em comparação com seus vizinhos. Pontos em regiões de baixa densidade relativa recebem LOF alto. Detecta outliers locais que métodos globais perdem.

One-Class SVM: Aprende a fronteira da região de normalidade no espaço de features. Pontos fora da fronteira são classificados como anomalias.

COPOD (*Copula-based Outlier Detection*): Usa cópulas para modelar dependências multivariadas sem assumir normalidade. Computacionalmente eficiente.

5.3 Como Tratar Outliers

Tabela 2: Estratégias de tratamento de outliers

Estratégia	Descrição	Quando usar	Cuidado
Remoção	Excluir registros com outliers	Erros claros de coleta; modelo sensível a extremos	Pode perder dados raros mas legítimos
Winsorização	Substituir por $Q_1 - 1.5 \cdot IQR$ ou $Q_3 + 1.5 \cdot IQR$	Quando os extremos são improváveis mas não impossíveis	Distorce a distribuição original
Transformação	Log, raiz quadrada, Box-Cox	Distribuição com cauda longa	Dificulta a interpretação dos coeficientes
Manter	Não tratar	Outliers genuínos (fraude, doença rara)	Algoritmos sensíveis (KNN, SVM linear) podem ser prejudicados
Modelar separadamente	Tratar outliers como classe separada	Detecção de anomalias	Requer volume suficiente de outliers

6 Normalização e Escalonamento

A maioria dos algoritmos de aprendizado de máquina é sensível à escala das features. Distâncias euclidianas (usadas em KNN, K-Means, SVM), regularizações (LASSO, Ridge) e gradientes (Redes Neurais) são afetados quando features têm escalas muito diferentes. O escalonamento é fundamental para garantir que todas as features contribuam equitativamente.

6.0.1 Min-Max Normalization

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1]$$

Mapeia todos os valores para o intervalo $[0, 1]$ (ou $[a, b]$ com ajuste). Muito sensível a outliers, que “comprimem” todos os outros valores.

6.0.2 Z-Score Standardization

$$x'_i = \frac{x_i - \mu}{\sigma}$$

Centraliza os dados com média 0 e desvio padrão 1. Assume que a distribuição é aproximadamente normal. O método mais usado como padrão.

6.0.3 Robust Scaler

$$x'_i = \frac{x_i - Q_2}{IQR}$$

Usa a mediana (Q_2) e o IQR. Robusto a outliers, pois não usa a média nem o desvio padrão.

6.0.4 MaxAbs Scaler

$$x'_i = \frac{x_i}{\max(|x|)}$$

Escala cada feature pelo seu valor absoluto máximo. Preserva zeros (útil para dados esparsos).

6.0.5 Transformação Logarítmica

$$x'_i = \log(1 + x_i)$$

Reduz a assimetria de distribuições com cauda longa à direita (renda, preços, contagens). O +1 evita $\log(0)$.

7 Seleção de Atributos (Feature Selection)

A seleção de atributos é o processo de identificar e manter apenas as features relevantes para o problema, eliminando aquelas redundantes, irrelevantes ou que introduzem ruído. Benefícios incluem: redução de overfitting, melhoria de interpretabilidade, redução do tempo de treinamento e melhoria da performance do modelo em dados de alta dimensionalidade.

Tabela 3: Guia de seleção de métodos de escalonamento

Método	Use quando	Não use quando
Min-Max	Redes neurais; dados sem outliers; range fixo necessário	Dados com outliers significativos
Z-Score	Regressão linear/logística; SVM; PCA	Distribuições muito assimétricas
Robust	Dados com outliers que devem ser mantidos	Algoritmos que exigem $[0, 1]$
MaxAbs	Dados esparsos (matrizes TF-IDF); quando zero é significativo	Dados densos com outliers
Log	Features com distribuição log-normal; contagens; preços	Dados com zeros (usar $\log(1 + x)$)

7.1 Métodos de Filtro

Avaliam a relevância de cada feature independentemente do algoritmo de aprendizado, usando medidas estatísticas:

- **Correlação de Pearson:** Para relações lineares entre variáveis numéricas. $r = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$. Remove features com $|r| > 0.9$ (altamente correlacionadas entre si).
- **V de Cramér:** Para associação entre variáveis categóricas. Baseado no teste chi-quadrado: $V = \sqrt{\chi^2 / (n \cdot \min(r - 1, c - 1))}$, onde r e c são o número de linhas e colunas da tabela de contingência.
- **ANOVA F-test:** Para relação entre feature numérica e alvo categórico. Testa se as médias da feature diferem significativamente entre as classes.
- **Informação Mútua:** Mede a dependência estatística geral (não apenas linear) entre a feature e o alvo: $I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x,y)}{p(x)p(y)}$.
- **Teste Chi-quadrado:** Para features categóricas vs. alvo categórico. Testa independência estatística.

7.2 Métodos Wrapper

Usam um algoritmo de aprendizado para avaliar subconjuntos de features. Mais poderosos que filtros, mas computacionalmente mais caros:

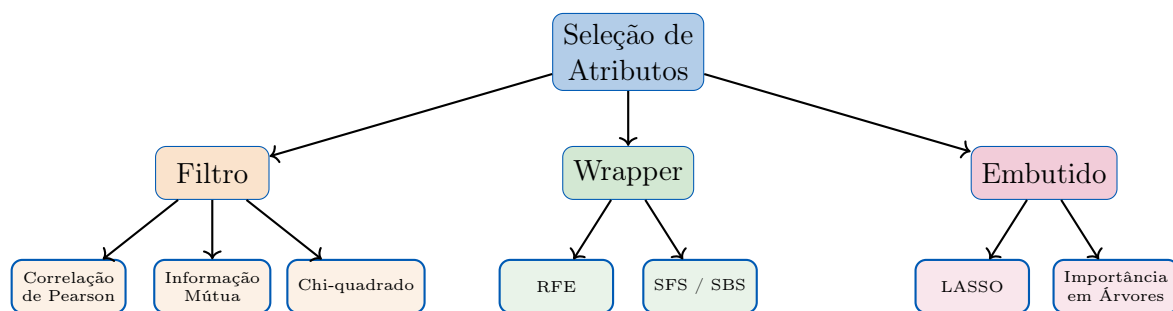
- **RFE** (*Recursive Feature Elimination*): Treina o modelo, remove a feature menos importante e repete até atingir o número desejado de features. Disponível em sklearn como `RFE(estimator, n_features_to_select)`.

- **Sequential Forward Selection (SFS):** Começa com conjunto vazio e adiciona a feature que mais melhora o desempenho em cada passo.
- **Sequential Backward Selection (SBS):** Começa com todas as features e remove a que menos prejudica o desempenho em cada passo.

7.3 Métodos Embutidos (Embedded)

A seleção acontece durante o próprio treinamento do modelo:

- **Regularização LASSO** (Tibshirani, 1996): Adiciona penalidade ℓ_1 ao custo: $\min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|_1$. Produz coeficientes exatamente zero para features irrelevantes, realizando seleção automática.
- **Importância em Árvores:** Florestas aleatórias e gradient boosting calculam a importância de cada feature pela redução total de impureza de Gini causada por ela. Também disponível como *permutation importance* (mais robusto, mas mais lento).



8 Redução de Dimensionalidade

8.1 PCA – Análise de Componentes Principais

O PCA (*Principal Component Analysis*) é uma técnica linear que projeta os dados em um espaço de menor dimensão, maximizando a variância retida. É amplamente usado para visualização, remoção de correlações e compressão de dados.

Derivação matemática:

Passo 1: Centralizar a matriz de dados: $\tilde{X} = X - \bar{X}$ (subtrair a média de cada coluna)

Passo 2: Calcular a matriz de covariância: $\Sigma = \frac{1}{n-1} \tilde{X}^T \tilde{X} \in \mathbb{R}^{p \times p}$

Passo 3: Calcular os autovetores e autovalores: $\Sigma v_i = \lambda_i v_i$, onde $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$

Passo 4: Ordenar os autovetores por autovalores decrescentes (os autovetores formam os eixos dos componentes principais)

Passo 5: Selecionar k componentes e projetar: $Z = \tilde{X} V_k$, onde $V_k \in \mathbb{R}^{p \times k}$ contém os k primeiros autovetores

Passo 6: Variância explicada por cada componente: $EVR_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$

Regra prática: Selecionar o menor k tal que $\sum_{i=1}^k EVR_i \geq 0.95$ (95% da variância explicada).

8.2 t-SNE e UMAP

t-SNE (*t-distributed Stochastic Neighbor Embedding* - Maaten & Hinton, 2008) e UMAP (*Uniform Manifold Approximation and Projection* - McInnes et al., 2018) são técnicas não lineares para **visualização** em 2D ou 3D. Preservam a estrutura de vizinhança local dos dados. Não devem ser usadas para feature reduction em produção (os eixos resultantes não têm interpretação e não podem ser aplicados a novos dados de forma consistente). UMAP é significativamente mais rápido que t-SNE e escalável para grandes datasets.

8.3 Autoencoders para Redução

Redes neurais do tipo autoencoder aprendem uma representação comprimida (*latent space*) dos dados. O encoder mapeia $x \rightarrow z$ (compressão) e o decoder mapeia $z \rightarrow \hat{x}$ (reconstrução). Ao treinar para minimizar $\|x - \hat{x}\|^2$, o encoder aprende features latentes significativas. Variational Autoencoders (VAEs) adicionam restrições probabilísticas ao espaço latente.

9 Balanceamento de Classes

O desbalanceamento de classes ocorre quando uma ou mais classes têm muito menos exemplos que as demais. Modelos treinados em dados desbalanceados tendem a favorecer a classe majoritária, resultando em alta acurácia global mas péssimo desempenho na classe minoritária (que frequentemente é a mais importante, como fraude ou diagnóstico positivo).

💡 Exemplo: Detecção de Fraude

Em um dataset de transações com 0.1% de fraudes: um modelo que sempre prevê “não é fraude” tem 99.9% de acurácia mas é completamente inútil. As métricas adequadas para dados desbalanceados são Precision, Recall, F1-Score e AUC-ROC.

9.0.1 Oversampling

SMOTE (*Synthetic Minority Over-sampling Technique* - Chawla et al., 2002): Gera exemplos sintéticos da classe minoritária interpolando entre instâncias existentes:

$$x_{new} = x_i + \lambda \cdot (x_{nn} - x_i), \quad \lambda \sim \mathcal{U}[0, 1]$$

onde x_{nn} é um vizinho aleatório de x_i no espaço de features.

ADASYN (*Adaptive Synthetic Sampling*): Variante do SMOTE que gera mais exemplos sintéticos nas regiões de decisão difíceis (onde a maioria dos vizinhos é da classe oposta).

Borderline-SMOTE: Foca na geração de sintéticos apenas nas instâncias próximas à fronteira de decisão, evitando amplificar ruído em regiões seguras.

9.0.2 Undersampling

Random Undersampling: Remove aleatoriamente instâncias da classe majoritária. Rápido mas pode eliminar informações úteis.

Tomek Links: Remove instâncias da classe majoritária que estão muito próximas à fronteira de decisão (pares Tomek), limpando a fronteira.

ENN (*Edited Nearest Neighbors*): Remove instâncias cuja classe difere da maioria dos seus k vizinhos, remove instâncias “ruidosas” de qualquer classe.

Tabela 4: Estratégias de balanceamento de classes

Estratégia	Vantagens	Desvantagens	Quando Usar
SMOTE	Aumenta diversidade; não perde dados	Pode gerar regiões irreais	Dataset pequeno a médio
ADASYN	Foca nas regiões difíceis	Amplifica ruído	Fronteira de decisão complexa
Random Under	Simples e rápido	Perde informação	Dataset grande; custo computacional alto
Tomek Links	Limpa a fronteira	Efeito modesto	Pré-processamento complementar

10 Estado da Arte

TabPFN (2022) – Transformers para Dados Tabulares

TabPFN (Prior-Fitted Network - Hollmann et al., 2022) é um modelo transformer pré-treinado especificamente para classificação em dados tabulares pequenos (< 1000 amostras, < 100 features). Em benchmarks, supera XGBoost e Random Forest em datasets pequenos, sem necessidade de ajuste de hiperparâmetros. Representa uma nova geração de modelos fundacionais para ciência de dados.

Deep Learning para Imputação

GAIN (*Generative Adversarial Imputation Nets* - Yoon et al., 2018) usa GANs para imputação: o gerador produz imputações e o discriminador distingue entre valores observados e imputados. GRAPE (*Graph-based Prediction of Missing Data* - You et al., 2020) modela o problema de imputação como predição em grafos bipartidos. São abordagens de ponta para dados altamente correlacionados.

 Engenharia Automática de Features

Featuretools (2018) usa “*deep feature synthesis*” para criar automaticamente features agregadas a partir de conjuntos de dados relacionais. AutoFeat usa regressão linear esparsa com transformações não lineares. OpenFE (Zhang et al., 2023) é um framework de geração automática de features de alto desempenho compatível com XGBoost e LightGBM.

11 Exercícios

- Exercício 1. (Mecanismos de Ausência)** Para cada situação, identifique o mecanismo de ausência (MCAR, MAR ou MNAR) e justifique: (a) Em uma pesquisa salarial, pessoas com salários acima de R\$20.000 tendem a não responder o campo de renda. (b) Em registros médicos, exames de certos dias da semana não foram digitalizados por falha do sistema. (c) Em um questionário online, usuários de dispositivos móveis não respondem questões com tabelas.
- Exercício 2. (Estratégia de Imputação)** Um dataset de churn de clientes possui 35% de valores ausentes na coluna “número de ligações ao suporte” e o diagnóstico indica mecanismo MAR. Escolha e justifique a melhor estratégia de imputação. Descreva como você validaria se a imputação foi bem-sucedida.
- Exercício 3. (Outliers)** Dado um conjunto de valores de renda mensal (R\$): [1200, 1500, 1800, 2100, 1900, 2300, 52000, 1600, 1400, 2000]. Aplique o método IQR para identificar outliers. Calcule Q1, Q3 e IQR manualmente. O valor R\$52.000 é um outlier? O que você faria com este valor em um modelo de crédito?
- Exercício 4. (Normalização)** Explique por que é necessário escalonar as features antes de aplicar: (a) K-Means; (b) PCA; (c) Árvores de Decisão; (d) Gradient Boosting. Em quais casos o escalonamento NÃO é necessário e por quê?
- Exercício 5. (Pipeline)** Implemente um pipeline sklearn completo para um dataset com 5 features numéricas (algumas com valores ausentes) e 3 features categóricas. O pipeline deve: tratar ausentes, escalar numéricas, codificar categóricas e treinar um classificador. Avalie com cross-validation e apresente as métricas.
- Exercício 6. (Desbalanceamento)** Um dataset de detecção de fraude tem 98% de transações normais e 2% de fraudes. Treine um modelo sem balanceamento e com SMOTE. Compare os resultados usando F1-Score, Precision, Recall e AUC-ROC. Por que a acurácia não é uma boa métrica para este problema?

12 Referências Bibliográficas

Referências

- [1] GARCÍA, S. et al. *Data Preprocessing in Data Mining*. Heidelberg: Springer, 2015.
- [2] CHAWLA, N. V. et al. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, v. 16, p. 321–357, 2002.
- [3] LIU, F. T.; TING, K. M.; ZHOU, Z.-H. Isolation Forest. In: *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, p. 413–422.
- [4] BREUNIG, M. M. et al. LOF: Identifying Density-Based Local Outliers. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, p. 93–104.
- [5] VAN BUUREN, S. *Flexible Imputation of Missing Data*. 2. ed. Boca Raton: CRC Press, 2018. Disponível em: <https://stefvanbuuren.name/fimd/>.
- [6] McINNES, L.; HEALY, J.; MELVILLE, J. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- [7] TIBSHIRANI, R. Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B*, v. 58, n. 1, p. 267–288, 1996.
- [8] VAN BUUREN, S.; GROOTHUIS-OUDSHOORN, K. mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software*, v. 45, n. 3, p. 1–67, 2011.
- [9] STEKHOVEN, D. J.; BUHLMANN, P. MissForest—non-parametric missing value imputation for mixed-type data. *Bioinformatics*, v. 28, n. 1, p. 112–118, 2012.
- [10] YOON, J.; JORDON, J.; SCHAAR, M. GAIN: Missing Data Imputation using Generative Adversarial Nets. In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, p. 5689–5698.
- [11] TUKEY, J. W. *Exploratory Data Analysis*. Reading: Addison-Wesley, 1977.
- [12] HOLLMANN, N. et al. TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second. *arXiv preprint arXiv:2207.01848*, 2022.